Linux System Administration (E1ITA307)

Linux Infrastructure Deployment Project Report

On

CENTRALIZED BACKUP AND RECOVERY SERVER

Submitted by

MANYATA (2411021061093)

HINDUSHREE (2411021061087)

ANUSHREE J (2411021061348)

CHASMA V (2411021061465)

In partial fulfillment of the award of the degree of

BACHELOR OF TECHNOLOGY

In

Information Technology

Under the Supervision of

Prof. Kusuma

ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY

BENGALURU

Oct 2026

Linux System Administration E1ITA307



DEPARTMENT OF INFORMATION TECHNOLOGY
ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the Project work entitled **"Centralized Backup and Recovery Server"** submitted by Manyata [2411021061093], HinduShree [2411021061087], Anushree J [2411021061348] and Chasma V [2411021061465] in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology - IT - DA of Alliance University, is a bonafide work during the academic year 2026-2027.

Prof. Kusuma	Dr. Ananthanagu U	Dr. Shekhar R
Course Handling Faculty	Course Coordinator	Professor & HoD

Table of Contents

1. Introduction

1.1 Aim

1.2 Background

1.3 Problem Statement

1.4 Objectives

2. Methodology

2.1 Tools and Technologies Used

2.2 Approach / Development Process

3. Linux System Administration – Design and Implementation

3.1 System Architecture

3.2 Linux Environment Setup

3.3 User and Group Management

3.4 File and Directory Management

3.5 Package and Process Management

3.6 Storage and File System Configuration

3.7 Backup and Automation

3.8 Security and Access Control

4. Results and Testing

4.1 Implementation Results

4.2 Testing and Validation

5. Conclusion and Future Scope

References

Appendices (A: Architecture, B: Commands, C: Shell Scripts)

1. Introduction

1.1 Aim

The main aim of this mini project is to design, develop and implement a **Centralized Backup and Recovery Server** on Linux. The system provides one central server that creates compressed backups of several client systems or directories, stores them in a single repository, verifies their integrity with checksums, restores them on demand, removes outdated backups according to a retention policy, and keeps a complete log of every operation.

The whole solution is implemented only with Bash shell scripting and standard Linux commands. Administrators work through a user-friendly, menu-driven interface in the terminal, and Cron is used to run backups automatically every day.

1.2 Background

Data loss can occur because of accidental deletion, disk failure, software faults, malware or human error. In an organisation with many servers, taking backups separately on every machine is difficult to manage: backups are stored in different places, are rarely verified and are easily forgotten. A centralized backup server solves this by collecting the backups of all systems at one place under one policy.

Linux provides powerful tools for this purpose: `tar` and `gzip` for archiving and compression, `sha256sum` for integrity checking, `rsync` and `ssh` for secure transfer from remote hosts, `find` for retention clean-up and `cron` for scheduling. This project combines these tools into one complete, menu-driven system.

1.3 Problem Statement

Manual backups are time-consuming, inconsistent and error-prone. Backups that are never verified may turn out to be corrupted only when they are needed, and old backups that are never deleted slowly fill the disk. Administrators also lack a single place that shows which clients are protected, when the last backup was taken and whether any operation failed.

Therefore, this project develops a centralized backup and recovery server that registers clients, performs backups with disk-space checks, verifies every backup with a SHA-256 checksum, restores data safely, enforces a retention policy, records all activity in a log, and handles invalid inputs and errors gracefully.

1.4 Objectives

- To provide a menu-driven terminal interface for all backup and recovery operations.
- To register and manage multiple backup clients (local directories and remote SSH hosts).
- To create compressed backups in one central repository using tar and gzip.
- To store a SHA-256 checksum with every backup and verify integrity before restoring.
- To restore backups safely into a separate recovery location without overwriting data.
- To apply a retention policy that deletes backups older than a configured number of days.
- To monitor repository storage usage and refuse backups when free space is too low.
- To automate daily backups using Cron and maintain logs and summary reports.
- To validate user input and handle errors with clear messages.

2. Methodology

2.1 Tools and Technologies Used

2.1.1 Software Requirements

- **Ubuntu Linux:** operating system used for development and testing.
- **Bash Shell:** used to write the menu, backup, restore, verification, retention and reporting scripts.
- **tar and gzip:** create and extract compressed backup archives.
- **sha256sum:** generates and checks checksums for backup integrity.
- **rsync and ssh:** transfer data from remote clients (optional remote mode).
- **Cron:** schedules the automatic daily backup.
- **Linux utilities:** df, du, find, ls, grep, awk, cut, sort, nl, tail, wc, date, mktemp, chmod, chgrp, useradd, groupadd.
- **Text editor:** used to create and modify scripts and configuration files.
- **Log and configuration files:** backup.log, clients.list and server.conf.

2.1.2 Hardware Requirements

- A laptop or desktop computer (or a virtual machine) running Ubuntu Linux.
- A processor and RAM sufficient to run Ubuntu and compress data.
- Adequate disk space for the backup repository – ideally on a separate disk or partition from the data being protected.

2.1.3 Other Tools and Resources

- **Linux Terminal:** used to run the menu and observe output.
- **Configuration file:** config/server.conf stores the repository path, retention days and minimum free-space percentage.
- **Client register:** config/clients.list stores each client's name, type and source path.
- **Cron logs and backup.log:** used to verify scheduled and manual operations.

2.2 Approach / Development Process

The project was developed in eight stages covering problem identification, requirement analysis, environment setup, configuration, implementation, automation, testing and documentation.

Step 1: Problem Identification

The need for a single server that automates backup, verification and recovery for several clients was identified.

Step 2: Requirement Analysis

Requirements were listed: Ubuntu Linux, Bash, tar, gzip, sha256sum, Cron, a menu interface, input validation, logging, and permissions that restrict access to backups.

Step 3: Linux Environment Setup

A project directory with separate folders for scripts, libraries, configuration, logs, reports, repository and recovery area was created.

Step 4: System Configuration

server.conf was written with the repository path, retention period and free-space limit. A user (backupop) and group (backupadmins) were created and the repository was restricted to that group.

Step 5: Implementation

One main script (bin/backup_server.sh) displays the menu and calls function modules in lib/: common.sh, clients.sh, backup.sh, restore.sh and maintenance.sh.

Step 6: Automation

Option 12 installs a Cron job that calls the main script with the --all argument, so every client is backed up daily without the menu.

Step 7: Testing

Each menu option was run with valid data and with invalid inputs. A backup was deliberately corrupted to confirm that verification detects it, and a restored copy was compared with the original using diff.

Step 8: Documentation

The structure, scripts, outputs, test cases and results were documented in this report.

3. Linux System Administration – Design and Implementation

3.1 System Architecture

The Centralized Backup and Recovery Server is a Bash-based system that runs on one Ubuntu Linux machine, the *backup server*. Clients are registered in a client list. When a backup is requested, the server checks free space, archives the client's data with tar and gzip, saves the archive in the repository under the client's own folder, computes a SHA-256 checksum, and writes the result to the log. Restoration verifies the checksum first and then extracts the archive into a safe location.

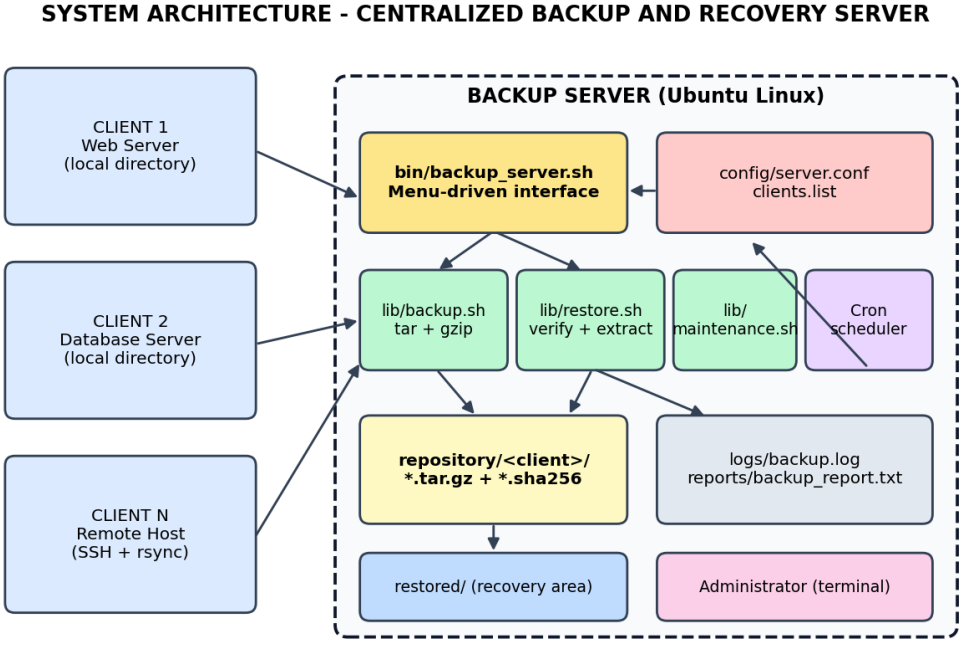


Figure 3.1: System Architecture of the Centralized Backup and Recovery Server

Components and their roles

- **Administrator:** runs the menu, selects clients and backups, and reviews logs and reports.
- **Backup Server:** Ubuntu Linux machine that hosts the scripts, repository, logs and the Cron service.
- **Clients:** directories (local) or remote hosts (SSH) whose data is protected.
- **Main script:** bin/backup_server.sh shows the menu and dispatches each choice to a function.
- **Function modules:** lib/ files containing registration, backup, restore, verification, retention, reporting and scheduling functions.
- **Repository:** repository/<client>/ holds the .tar.gz archives and their .sha256 checksum files.
- **Logs and reports:** logs/backup.log records every operation; reports/backup_report.txt is the summary report.
- **Cron:** runs the scheduled daily backup.

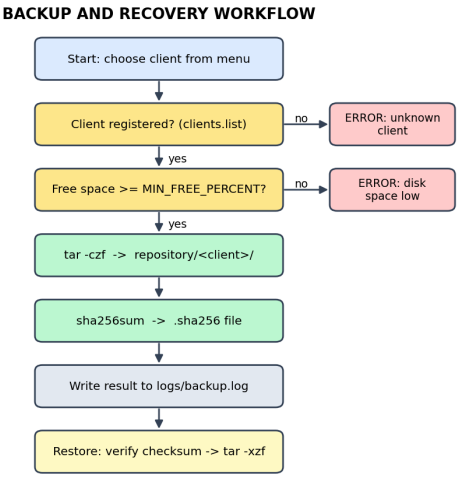


Figure 3.2: Backup and recovery workflow

3.2 Linux Environment Setup

3.2.1 Linux Distribution and Version

The project was developed and tested on Ubuntu Linux (Ubuntu 24.04 LTS). The distribution and the filesystem holding the repository were checked with `lsb_release -a`, `df -h /` and `df -i /`.

```
$ lsb_release -a
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.4 LTS
Release:        24.04
Codename:        noble
$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/vda        252G  8.6G   10G  47% /
$ df -i /
Filesystem       Inodes  IUsed    IFree IUse% Mounted on
/dev/vda       16777216 194038 16583178    2% /
```

Figure 3.3: Ubuntu version and filesystem details (sample output)

3.2.2 Installation Process

The project directory `centralized-backup-server` was created with `mkdir -p` and contains separate folders for scripts, libraries, configuration, logs, reports, the repository, the recovery area, tests and documentation. No software installation is needed beyond the default Ubuntu tools; `rsync` is required only for remote clients (`sudo apt install rsync`).

```
centralized-backup-server/
|-- bin/
|   `-- backup_server.sh        (main menu script)
|-- lib/
|   |-- common.sh   clients.sh   backup.sh
|   `-- restore.sh  maintenance.sh
|-- config/
|   |-- server.conf             (repository, retention, free-space limit)
|   `-- clients.list            (registered clients)
|-- repository/                 (central backup storage)
|-- restored/                   (recovery area)
|-- logs/       backup.log
|-- reports/    backup_report.txt
|-- tests/      docs/
```

Figure 3.4: Project directory structure

3.2.3 Hostname Configuration

The hostname identifies the backup server on the network and in log files. It is checked with `hostname` and `hostnamectl`. A meaningful name such as *backup-server* is recommended for a real deployment.

3.2.4 Network Configuration

Local-directory backups do not need a network. Remote clients are reached over SSH, so the backup server must be able to reach them; interfaces and routes are inspected with `ip address` and `ip route`. For unattended remote backups SSH key authentication should be configured, because the script uses `BatchMode=yes` and never waits for a password.

3.2.5 User Accounts

The scripts run under the administrator's account. For access control a dedicated operator account **backupop** and a group **backupadmins** were created (see Section 3.3).

3.2.6 Required Packages

The system uses only utilities that are present in a default Ubuntu installation: bash, tar, gzip, coreutils (sha256sum, df, du, date, mktemp), findutils, grep, gawk and cron. Versions used during testing: GNU bash 5.2.21, GNU tar 1.35 and sha256sum from GNU coreutils 9.4.

3.2.7 System Services

The Cron service must be active for scheduled backups. It is checked with `systemctl is-active cron` and enabled at boot with `sudo systemctl enable --now cron`.

3.3 User and Group Management

Backups contain sensitive data, so access to the repository must be limited to authorised administrators. A group and a user were created for this purpose.

3.3.1 User Creation

```
$ sudo useradd -m -s /bin/bash backupop
$ sudo passwd backupop
```

3.3.2 Group Creation

```
$ sudo groupadd backupadmins
```

3.3.3 User-Group Assignment

The operator account was added to the group with `usermod -aG` and the membership was verified with `id` and `getent`.

```
$ sudo usermod -aG backupadmins backupop
$ id backupop
uid=1001(backupop) gid=1003(backupop) groups=1003(backupop),1002(backupadmins)
$ getent group backupadmins
backupadmins:x:1002:backupop
```

Figure 3.5: User and group configuration (sample output)

3.3.4 Password Management

Passwords are set with `sudo passwd backupop` and are stored only as hashes in /etc/shadow. They are never displayed or written by the scripts. Password status can be checked with `sudo passwd -S backupop`.

3.3.5 File Ownership

The repository directory is assigned to the *backupadmins* group with `chgrp -R backupadmins repository`, so that only that group can work with stored backups.

3.3.6 File Permissions

The repository directory uses mode **2770**: the owner and group have full access, others have none, and the set-group-ID bit makes new files inherit the group. The backup script sets mode **640** on every archive and checksum file, so the owner can read and write, the group can only read, and others have no access.

```
$ ls -ld repository
drwxrws--- 4 root backupadmins 4096 Oct 15 01:59 repository
$ ls -l repository/webserver
-rw-r----- 1 root backupadmins 236 Oct 15 01:59 webserver_20261015_015926.tar.gz
-rw-r----- 1 root backupadmins  99 Oct 15 01:59 webserver_20261015_015926.tar.gz.sha256
```

Figure 3.6: Repository ownership and permissions (sample output)

3.3.7 Access Control

Users outside the group cannot list or read the repository. Members of *backupadmins* can. Access is tested with `sudo -u backupop ls repository` (allowed) and the same command as an unrelated user (permission denied).

3.4 File and Directory Management

3.4.1 Directory Structure

Each part of the system has its own folder (Figure 3.4). Backups of every client are kept in their own sub-folder of the repository, `repository/<client>/`, which keeps the repository organised and easy to search.

3.4.2 File Creation and Management

Archives are named `<client>_YYYYMMDD_HHMMSS.tar.gz` using the `date` command, so every backup has a unique, sortable name. The configuration is a plain text file read with `source`, and the client list uses a simple `name|type|source` format.

3.4.3 File Permissions and Ownership

Scripts are made executable with `chmod +x bin/*.sh`. Archives and checksums are protected with `chmod 640` as described in Section 3.3.

3.4.4 Symbolic Links

A symbolic link can give the latest backup a fixed name, for example `ln -sf webserver_20261015_015936.tar.gz repository/webserver/latest.tar.gz`. The link provides an alternative path without a second copy of the data.

3.4.5 Disk Usage Monitoring

`df -P` reports the free space of the repository filesystem and `du -sh` reports the size of each client's backups. Menu option 9 shows both.

3.4.6 File Searching and Log Filtering

`find` locates archives older than the retention period, `grep -c` counts successful and failed operations in the log, and `tail` shows recent log entries.

3.5 Package and Process Management

3.5.1 Package Installation and Removal

Packages are managed with APT. Only `rsync` (remote clients) and `cron` may need installation on a minimal system.

```
$ sudo apt update
$ sudo apt install rsync cron
$ dpkg -s rsync | grep -E 'Status|Version'
```

3.5.2 Dependency Management

The scripts check for dependencies before use: `command -v rsync` and `command -v crontab` confirm that a tool exists, and a clear message tells the administrator which package to install when it is missing. APT resolves package dependencies automatically.

3.5.3 Software Updates

```
$ sudo apt update
$ apt list --upgradable
$ sudo apt upgrade
```

3.5.4 Process Monitoring

While a large backup runs, the tar process can be observed with `ps aux | grep '[t]ar -czf'` and `top`. Memory availability is checked with `free -h`.

```
$ free -h
               total        used        free      shared  buff/cache   available
Mem:           3.9Gi       190Mi       3.8Gi        14Mi        62Mi       3.7Gi
```

Figure 3.7: Memory status (sample output)

3.5.5 Process Termination

A runaway backup can be stopped by finding its PID with `pgrep -f backup_server.sh` and sending `kill <PID>`. A partial archive is not left behind in normal use because the script removes a failed archive.

3.5.6 Service Management

The Cron service is managed with systemctl.

```
$ systemctl status cron --no-pager
$ systemctl is-active cron
$ sudo systemctl enable --now cron
```

3.5.7 Resource Monitoring

Resource monitoring in this project concentrates on repository storage: free space percentage is calculated by the helper function `free_percent` and compared with `MIN_FREE_PERCENT` before every backup.

3.6 Storage and File System Configuration

3.6.1 Disk and Partition Configuration

`lsblk` and `df` show the available block devices and mounted filesystems. In a production setup the repository should be on a separate disk or partition from the data being protected, so that a failure of the production disk does not also destroy the backups.

3.6.2 File System Creation

The project uses an existing ext4 filesystem. If a dedicated backup disk is added, it can be prepared with `sudo mkfs.ext4 /dev/sdb1` and the repository path in server.conf changed to its mount point.

3.6.3 Mounting and Unmounting

```
$ sudo mkdir -p /backup
$ sudo mount /dev/sdb1 /backup
$ findmnt /backup
$ sudo umount /backup
```

3.6.4 Logical Volume Management (LVM)

LVM is not required by the current project. If the repository later needs to grow, a logical volume can be extended with `lvextend` and `resize2fs`. Existing volumes are listed with `sudo pvs`, `vgs` and `lvs`.

3.6.5 Storage Allocation

Repository usage is shown by menu option 9, which combines `df -h` for overall capacity with `du -sh` per client.

3.6.6 Disk Usage Monitoring

Before each backup the script reads the used percentage with `df -P | awk`, converts it to free percentage and refuses to continue if it is below the configured limit. The settings in config/server.conf are:

```
BACKUP_ROOT="$BASE_DIR/repository"
RETENTION_DAYS=7
MIN_FREE_PERCENT=10
```

3.6.7 Storage Architecture and Configuration Details

The storage layout consists of one repository directory with one sub-directory per client. Each backup is a pair of files: the compressed archive and its checksum file. A recovery area (`restored/`) receives extracted data so that live data is never overwritten.

3.7 Backup and Automation

3.7.1 Backup Strategy

Every backup is a full backup compressed with `tar -czf`. For a local client the script archives the source directory directly. For a remote client it first copies the data to a temporary staging directory using `rsync -a` over SSH, archives that copy and then deletes the staging area. After the archive is created a SHA-256 checksum is stored beside it.

```
tar -czf "$archive" -C "$(dirname "$src")" "$(basename "$src")"
cd "$dest" && sha256sum "$archive_name" > "$archive_name.sha256"
```

3.7.2 Backup Location

Backups are stored in `repository/<client>/` under the project directory (or the path set in BACKUP_ROOT).

3.7.3 Backup Frequency

Backups can be run manually from the menu at any time (options 3 and 4) and automatically once a day through Cron (option 12). The hour is chosen by the administrator.

3.7.4 Shell Script Automation

All logic is written in Bash. The main script supports two modes: the interactive menu, and the `--all` argument that backs up every client without any interaction, which is what Cron uses.

3.7.5 Cron Configuration

Option 12 validates the hour (0-23), checks that crontab exists, refuses to install a duplicate and appends this line to the user's crontab:

```
0 2 * * * /home/user/centralized-backup-server/bin/backup_server.sh --all >> /home/user/centralized-backup-server/logs
```

The example above is the entry created when the hour 2 is chosen. It is verified with `crontab -l`.

3.7.6 Log Generation

Every operation is appended to `logs/backup.log` in the format *date* | *level* | *message*, with the levels INFO, SUCCESS and ERROR.

```
2026-10-15 01:59:36 | SUCCESS | Backup created: webserver_20261015_015936.tar.gz size=4.0K
2026-10-15 01:59:36 | INFO    | Verification passed: webserver_20261015_015936.tar.gz
2026-10-15 01:59:36 | SUCCESS | Restored webserver_20261015_015936.tar.gz -> .../restored/webserver_20261015_015936 (2
2026-10-15 01:59:36 | ERROR   | Verification failed: database_20261015_015926.tar.gz
2026-10-15 01:59:36 | INFO    | Retention removed 1 backup(s)
```

Figure 3.8: Entries from logs/backup.log (sample output)

3.7.7 Recovery Procedure

Menu option 7 performs recovery in five steps: select the client; select the backup; verify the archive (checksum and tar structure); choose a destination (default is a new time-stamped folder in `restored/`); extract with `tar -xzf` and report the number of files restored. Recovery is refused if the checksum does not match or the destination folder is not empty, so existing files are never overwritten.

3.7.8 Benefits of Automation

Automation removes the need to remember backups, makes them consistent, produces a log that proves they ran, and reduces the risk of missing a backup before a failure. The checksum check ensures that a damaged backup is detected before it is needed for recovery.

3.8 Security and Access Control

3.8.1 User Permissions

The repository is accessible only to the *backupadmins* group. The operator account is a member of that group and no other user can read stored backups.

3.8.2 File Permissions

Directory mode 2770 and file mode 640 (Section 3.3.6) enforce the principle of least privilege on the stored data.

3.8.3 Firewall Configuration

When remote clients are used, the firewall should allow only SSH. The status is checked with `sudo ufw status verbose`, and SSH can be allowed with `sudo ufw allow ssh` after reviewing the network requirements. Local-directory backups need no open network port.

3.8.4 SSH Configuration

Remote backups use SSH, which encrypts the data in transit. For unattended jobs, key-based authentication is set up with `ssh-keygen` and `ssh-copy-id user@host`. The script connects with `BatchMode=yes` and a 10-second timeout, so it fails with a clear message instead of hanging when a host is unreachable.

3.8.5 Service Access Control

Only the cron service is needed for automation. Its state is checked with `systemctl is-active cron`.

3.8.6 Password Policies

Account passwords follow the organisation's policy. The scripts never ask for or store passwords.

3.8.7 Security Monitoring

The log file records every verification failure and error, so tampering or corruption is visible. The report (option 11) counts successful and failed operations.

3.8.8 Security Benefits

- **Group-restricted repository:** only authorised administrators can access backups.

- **Checksums:** detect corruption or modification of a backup before recovery.
- **Safe restore:** never overwrites existing data and always verifies first.
- **Input validation:** client names and paths are checked, preventing malformed entries.
- **Logging:** every action leaves an audit trail.

4. Results and Testing

4.1 Implementation Results

The system was run with two sample clients: *webserver* (a small site with an HTML file and an Nginx configuration) and *database* (three SQL data files). Every menu option was executed. The outputs below were captured from the working scripts.

4.1.1 Menu-Driven User Interface

```
==============================================
   CENTRALIZED BACKUP AND RECOVERY SERVER
==============================================
  1) Register new client
  2) List registered clients
  3) Back up a client
  4) Back up ALL clients
  5) List backups in repository
  6) Verify backup integrity
  7) Restore a backup
  8) Apply retention policy (delete old backups)
  9) Show repository storage usage
 10) View recent logs
 11) Generate summary report
 12) Schedule daily automatic backup (cron)
  0) Exit
----------------------------------------------
Enter your choice [0-12]:
```

Figure 4.1: Main menu of the backup server

4.1.2 Client Registration

```
Client name (2-20 letters, digits, - or _): webserver
Client type:  1) Local directory   2) Remote host (SSH)
Choice [1-2]: 1
Absolute path of directory to back up: /tmp/demo/webserver
[OK]    Client 'webserver' registered.

----- Registered Clients -----
NAME           TYPE     SOURCE
-------------  --------  ------------------------------
webserver      local    /tmp/demo/webserver
database       local    /tmp/demo/database
```

Figure 4.2: Registering and listing clients

4.1.3 Backup and Repository Listing

```
[INFO]  Starting backup of 'webserver' (local: /tmp/demo/webserver)
[OK]    Backup created: webserver_20261015_015926.tar.gz (4.0K)
[INFO]  Starting backup of 'database' (local: /tmp/demo/database)
[OK]    Backup created: database_20261015_015926.tar.gz (4.0K)
Summary: 2 succeeded, 0 failed.

----- Backups in Central Repository -----
database     database_20261015_015926.tar.gz      4.0K  2026-10-15 01:59
webserver    webserver_20261015_015926.tar.gz     4.0K  2026-10-15 01:59
```

Figure 4.3: Backing up all clients and listing the repository

4.1.4 Verification and Restore

```
----- Restore Backup -----
[OK]    Integrity verified: webserver_20261015_015936.tar.gz
Restore destination [/home/.../restored/webserver_20261015_015936]:
[OK]    Restored 2 file(s) to: /home/.../restored/webserver_20261015_015936
```

Figure 4.4: Restoring a verified backup

The restored directory was compared with the original using `diff -r`; the command produced no differences, confirming that recovery is exact.

4.1.5 Storage Usage, Report and Retention

```
----- Repository Storage Usage -----
Filesystem      Size  Used Avail Use% Mounted on
/dev/vda        252G  8.6G   10G  47% /

CLIENT         BACKUPS  SIZE
database       1        12K
webserver      1        12K
Free space: 53%  (minimum required: 10%)
```

Figure 4.5: Repository storage usage

```
[OK]    Report saved: .../reports/backup_report.txt
========================================
  CENTRALIZED BACKUP AND RECOVERY REPORT
========================================
Generated      : 2026-10-15 01:59:36
Clients        : 2
Total backups  : 2
Repository size: 28K
Free space     : 53%
Successful ops : 5
Failed ops     : 2
```

Figure 4.6: Summary report generated by option 11

For retention, one backup was given a modification date ten days in the past with `touch -d '10 days ago'`. Option 8 detected it as older than the 7-day limit, asked for confirmation, and removed the archive together with its checksum file (*"[OK] 1 old backup(s) removed."*).

4.1.6 Error Handling

Invalid inputs and error conditions were tested deliberately. The script prints a clear message, writes errors to the log where relevant, and returns to the menu without crashing.

```
[ERROR] Invalid choice '99'. Please enter a number from 0 to 12.
[ERROR] Invalid choice 'abc'. Please enter a number from 0 to 12.
[ERROR] Invalid client name.                      (name "bad name!")
[ERROR] Client 'webserver' already exists.
[ERROR] '/no/such/dir' is not an existing absolute directory path.
[ERROR] Invalid choice '3'.                       (client type)
[ERROR] Client 'ghost' is not registered.
[ERROR] Invalid selection: '9'.                   (backup number)
[ERROR] Destination '/tmp/demo/webserver' is not empty. Choose another location.
```

Figure 4.7: Error messages for invalid input

4.1.7 Corruption Detection

To prove that verification works, extra bytes were appended to one archive. Both Verify (option 6) and Restore (option 7) detected the damage, and the restore was cancelled so that no bad data was extracted. The original archive was put back afterwards.

```
database_20261015_015926.tar.gz: FAILED
[ERROR] Checksum MISMATCH - database_20261015_015926.tar.gz is corrupted.
[ERROR] Restore cancelled (verification failed).
```

Figure 4.8: Corrupted backup detected

4.2 Testing and Validation

Test Case	Input / Action	Expected Result	Actual Result	Status
Menu navigation	Choices 0-12	Each option runs its function	All options ran correctly	Pass
Invalid menu choice	99, abc, empty	Error message, menu again	Error shown, menu redisplayed	Pass
Register client	Valid name and path	Client saved in clients.list	Client registered and listed	Pass
Invalid client name	"bad name!"	Rejected	Invalid client name	Pass
Duplicate client	Existing name	Rejected	Already exists message	Pass
Invalid source path	/no/such/dir	Rejected	Path error message	Pass
Unknown client	ghost	Rejected	Not registered message	Pass
Single backup	webserver	tar.gz + .sha256 created	Archive and checksum created	Pass
Backup all	Option 4	Every client backed up	2 succeeded, 0 failed	Pass
Verify backup	Intact archive	Integrity verified	Verified	Pass
Corruption detection	Modified archive	Failure reported	Checksum MISMATCH shown	Pass
Restore	Verified archive	Files restored, identical	diff -r showed no differences	Pass
Restore to non-empty dir	Existing folder	Refused	Destination not empty	Pass
Retention policy	Backup aged 10 days	Old backup deleted	1 old backup removed	Pass
Storage usage / report / logs	Options 9, 10, 11	Correct output	Output displayed, report saved	Pass
Cron scheduling	Option 12	Entry in crontab	Needs cron installed; verify with crontab -l	Verify on target VM
Remote (SSH) client	user@host:/path	Backup via rsync	Needs reachable host; code includes error handling	Verify on target VM

TABLE 1: TESTING AND VALIDATION

4.2.1 Testing Summary

All core operations – registration, backup, verification, restore, retention, storage usage, logging, reporting and error handling – behaved as expected with local clients. The Cron scheduling and remote SSH features depend on the cron service and a reachable remote host; they are implemented with validation and error handling and should be confirmed on the target Ubuntu machine during the demonstration.

5. Conclusion and Future Scope

5.1 Conclusion

The Centralized Backup and Recovery Server provided practical experience in Linux system administration. A single menu-driven Bash application now registers clients, creates compressed backups in one repository, protects them with SHA-256 checksums, restores them safely, applies a retention policy, monitors storage, records a full log and produces reports.

The project used Bash functions and modules, input validation, exit-status checking, conditional logic, loops, text processing with awk, cut and grep, file permissions, users and groups, and Cron scheduling. Testing with valid and invalid inputs, a deliberately corrupted archive and a restored-versus-original comparison showed that the main functions work and that errors are handled gracefully.

The project achieved its objective of providing a complete, functional, menu-driven backup and recovery solution using only Bash and Linux commands.

5.2 Future Scope

- **Incremental backups:** use rsync hard-link snapshots or tar incremental mode to save space and time.
- **Encryption:** encrypt archives with GPG before storing them.
- **Off-site copy:** replicate the repository to a second server or cloud storage.
- **E-mail alerts:** notify administrators when a backup fails or free space is low.
- **Database dumps:** include mysqldump or pg_dump for consistent database backups.
- **Web dashboard:** display client status, backup history and storage usage in a browser.
- **Automated restore tests:** periodically extract a sample backup to prove recoverability.
- **Configuration management:** deploy the server and clients with tools such as Ansible.

References

1. Ubuntu Documentation, Ubuntu Server Documentation. https://ubuntu.com/server/docs/

2. GNU Project, GNU Bash Reference Manual. https://www.gnu.org/software/bash/manual/bash.html

3. GNU Project, GNU Coreutils Manual. https://www.gnu.org/software/coreutils/manual/coreutils.html

4. GNU Project, GNU Tar Manual. https://www.gnu.org/software/tar/manual/tar.html

5. Ubuntu Manpages, crontab – maintains crontab files for individual users. https://manpages.ubuntu.com/

6. rsync Project, rsync Documentation. https://rsync.samba.org/documentation.html

7. Linux Manual Pages Project, Linux man pages. https://www.kernel.org/doc/man-pages/

Appendices

Appendix A: System Architecture

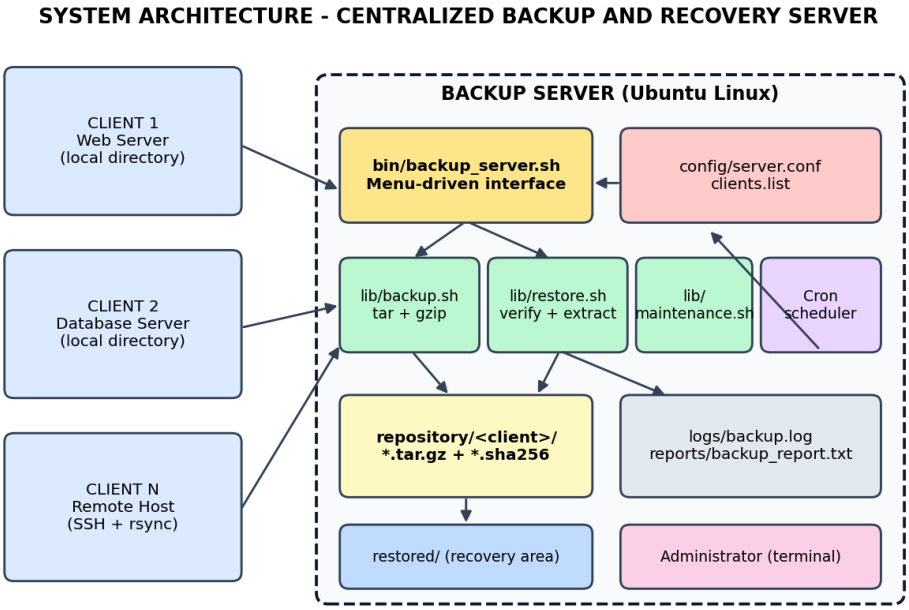


Figure A1: System Architecture of the Centralized Backup and Recovery Server

Appendix B: Linux Commands Used

Command	Purpose in this project
tar -czf / -xzf / -tzf	Create, extract and list compressed archives
sha256sum / sha256sum -c	Create and check backup checksums
df -P, df -h, du -sh	Free space and size of repository and backups
find -mtime +N -delete	Retention: select and delete old backups
grep, cut, awk, nl, sort, tail, wc	Process client list, logs and numbers
date, mktemp, mkdir -p	Time-stamped names, staging directory, folders
chmod, chgrp, ls -l	Permissions and ownership of the repository
useradd, groupadd, usermod, id, getent	User and group management
rsync, ssh	Fetch data from remote clients
crontab, systemctl	Scheduling and service status
diff -r	Compare restored data with original

APPENDIX C: SHELL SCRIPTS

C.1 Main Menu Script — bin/backup_server.sh

```
#!/bin/bash
# backup_server.sh - Centralized Backup and Recovery Server (main menu)
# Usage:  ./backup_server.sh          interactive menu
#         ./backup_server.sh --all    back up every client (used by cron)

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
LIB_DIR="$SCRIPT_DIR/../lib"

for f in common clients backup restore maintenance; do
    if [[ ! -f "$LIB_DIR/$f.sh" ]]; then
        echo "ERROR: missing module $LIB_DIR/$f.sh" >&2
        exit 1
    fi
    source "$LIB_DIR/$f.sh"
done

# Non-interactive mode for cron
if [[ "$1" == "--all" ]]; then
    log "INFO" "Scheduled backup started"
    backup_all
    exit $?
fi

show_menu() {
    clear 2>/dev/null
    echo "================================================"
    echo "   CENTRALIZED BACKUP AND RECOVERY SERVER"
    echo "================================================"
    echo "  1) Register new client"
    echo "  2) List registered clients"
    echo "  3) Back up a client"
    echo "  4) Back up ALL clients"
    echo "  5) List backups in repository"
    echo "  6) Verify backup integrity"
    echo "  7) Restore a backup"
    echo "  8) Apply retention policy (delete old backups)"
    echo "  9) Show repository storage usage"
    echo " 10) View recent logs"
    echo " 11) Generate summary report"
    echo " 12) Schedule daily automatic backup (cron)"
    echo "  0) Exit"
    echo "------------------------------------------------"
}

while true; do
    show_menu
    read -r -p "Enter your choice [0-12]: " choice || { echo; break; }
    echo
    case "$choice" in
        1)  register_client ;;
        2)  list_clients ;;
        3)  choose_client && backup_client "$SELECTED" ;;
        4)  backup_all ;;
        5)  list_backups ;;
        6)  verify_backup ;;
        7)  restore_backup ;;
        8)  apply_retention ;;
        9)  storage_usage ;;
        10) view_logs ;;
        11) generate_report ;;
        12) schedule_backup ;;
        0)  echo "Goodbye!"; log "INFO" "Session ended"; exit 0 ;;
        *)  msg_err "Invalid choice '$choice'. Please enter a number from 0 to 12." ;;
    esac
    echo
    pause
done
```

C.2 Common Functions — lib/common.sh

```
#!/bin/bash
# common.sh - shared settings and helper functions

BASE_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
CONFIG_FILE="$BASE_DIR/config/server.conf"
CLIENT_FILE="$BASE_DIR/config/clients.list"
LOG_FILE="$BASE_DIR/logs/backup.log"

if [[ ! -f "$CONFIG_FILE" ]]; then
    echo "ERROR: Configuration file not found: $CONFIG_FILE" >&2
    exit 1
fi
source "$CONFIG_FILE"

mkdir -p "$BACKUP_ROOT" "$BASE_DIR/logs" "$BASE_DIR/reports" "$BASE_DIR/restored"
touch "$CLIENT_FILE"

# Use colours only when the output is a terminal
if [[ -t 1 ]]; then
    GREEN=$'\e[32m'; RED=$'\e[31m'; YELLOW=$'\e[33m'; CYAN=$'\e[36m'; NC=$'\e[0m'
else
    GREEN=""; RED=""; YELLOW=""; CYAN=""; NC=""
fi

log()      { printf '%s | %-7s | %s\n' "$(date '+%Y-%m-%d %H:%M:%S')" "$1" "$2" >> "$LOG_FILE"; }
msg_ok()   { echo "${GREEN}[OK]${NC}    $1"; }
msg_err()  { echo "${RED}[ERROR]${NC} $1"; }
msg_warn() { echo "${YELLOW}[WARN]${NC}  $1"; }
msg_info() { echo "${CYAN}[INFO]${NC}  $1"; }

pause() { read -r -p "Press Enter to continue..." _; }

valid_name()    { [[ "$1" =~ ^[A-Za-z0-9_-]{2,20}$ ]]; }
client_exists() { grep -q "^$1|" "$CLIENT_FILE"; }
client_field()  { grep "^$1|" "$CLIENT_FILE" | cut -d'|' -f"$2"; }

# Print free space (percentage) of the filesystem holding the repository
free_percent() {
    local used
    used=$(df -P "$BACKUP_ROOT" | awk 'NR==2 {gsub(/%/,"",$5); print $5}')
    echo $((100 - used))
}

# Let the user choose a registered client; result stored in $SELECTED
choose_client() {
    SELECTED=""
    if [[ ! -s "$CLIENT_FILE" ]]; then
        msg_err "No clients registered. Use option 1 first."
        return 1
    fi
    echo "Registered clients:"
    cut -d'|' -f1 "$CLIENT_FILE" | nl -w2 -s') '
    read -r -p "Enter client name: " SELECTED
    if ! client_exists "$SELECTED"; then
        msg_err "Client '$SELECTED' is not registered."
        return 1
    fi
}

# Let the user choose a backup archive of $1; result stored in $ARCHIVE
choose_archive() {
    ARCHIVE=""
    local dir="$BACKUP_ROOT/$1"
    local files=()
    mapfile -t files < <(ls -1 "$dir"/*.tar.gz 2>/dev/null | sort)
    if [[ ${#files[@]} -eq 0 ]]; then
        msg_err "No backups found for client '$1'."
        return 1
    fi
    local i=1
    for f in "${files[@]}"; do
        printf '%2d) %s (%s)\n' "$i" "$(basename "$f")" "$(du -h "$f" | cut -f1)"
        i=$((i + 1))
    done
    read -r -p "Select backup number: " n
    if ! [[ "$n" =~ ^[0-9]+$ ]] || (( n < 1 || n > ${#files[@]} )); then
        msg_err "Invalid selection: '$n'."
        return 1
    fi
    ARCHIVE="${files[$((n - 1))]}"
}
```

C.3 Client Management — lib/clients.sh

```
#!/bin/bash
# clients.sh - client registration and listing
# clients.list format:  name|type|source     (type = local | remote)

register_client() {
    echo "----- Register New Client -----"
    read -r -p "Client name (2-20 letters, digits, - or _): " name
    if ! valid_name "$name"; then
        msg_err "Invalid client name."; return 1
    fi
    if client_exists "$name"; then
        msg_err "Client '$name' already exists."; return 1
    fi

    echo "Client type:  1) Local directory   2) Remote host (SSH)"
    read -r -p "Choice [1-2]: " t
    case "$t" in
        1)
            read -r -p "Absolute path of directory to back up: " src
            if [[ "$src" != /* || ! -d "$src" ]]; then
                msg_err "'$src' is not an existing absolute directory path."; return 1
            fi
            if [[ ! -r "$src" ]]; then
                msg_err "No read permission for '$src'."; return 1
            fi
            echo "$name|local|${src%/}" >> "$CLIENT_FILE"
            ;;
        2)
            read -r -p "Remote source (user@host:/path): " src
            if ! [[ "$src" =~ ^[A-Za-z0-9._-]+@[A-Za-z0-9.-]+:/.+$ ]]; then
                msg_err "Format must be user@host:/absolute/path"; return 1
            fi
            echo "$name|remote|$src" >> "$CLIENT_FILE"
            ;;
        *)
            msg_err "Invalid choice '$t'."; return 1 ;;
    esac
    log "INFO" "Client registered: $name"
    msg_ok "Client '$name' registered."
}

list_clients() {
    echo "----- Registered Clients -----"
    if [[ ! -s "$CLIENT_FILE" ]]; then
        msg_warn "No clients registered yet."; return 0
    fi
    printf '%-14s %-8s %s\n' "NAME" "TYPE" "SOURCE"
    printf '%-14s %-8s %s\n' "--------------" "--------" "------------------------------"
    while IFS='|' read -r n t s; do
        printf '%-14s %-8s %s\n' "$n" "$t" "$s"
    done < "$CLIENT_FILE"
}
```

C.4 Backup Functions — lib/backup.sh

```
#!/bin/bash
# backup.sh - create compressed backups in the central repository

backup_client() {
    local name="$1" type src dest ts archive stage rc=0
    if ! client_exists "$name"; then
        msg_err "Client '$name' is not registered."; return 1
    fi
    type=$(client_field "$name" 2)
    src=$(client_field "$name" 3)

    # 1. Check free space on the backup server
    local free
    free=$(free_percent)
    if (( free < MIN_FREE_PERCENT )); then
        msg_err "Only ${free}% free space (minimum ${MIN_FREE_PERCENT}%). Backup aborted."
        log "ERROR" "Backup of $name aborted: low disk space (${free}% free)"
        return 1
    fi

    dest="$BACKUP_ROOT/$name"
    mkdir -p "$dest"
    ts=$(date '+%Y%m%d_%H%M%S')
    archive="$dest/${name}_${ts}.tar.gz"

    msg_info "Starting backup of '$name' ($type: $src)"
    if [[ "$type" == "local" ]]; then
        if [[ ! -d "$src" ]]; then
            msg_err "Source '$src' no longer exists."
            log "ERROR" "Backup of $name failed: source missing"
            return 1
        fi
        tar -czf "$archive" -C "$(dirname "$src")" "$(basename "$src")" 2>/dev/null || rc=$?
    else
        if ! command -v rsync >/dev/null 2>&1; then
            msg_err "rsync is not installed (sudo apt install rsync)."; return 1
        fi
        stage=$(mktemp -d)
        if ! rsync -a -e "ssh -o BatchMode=yes -o ConnectTimeout=10" "$src/" "$stage/" 2>/dev/null; then
            rm -rf "$stage"
            msg_err "Could not reach remote client '$src'."
            log "ERROR" "Backup of $name failed: remote unreachable"
            return 1
        fi
        tar -czf "$archive" -C "$stage" . 2>/dev/null || rc=$?
        rm -rf "$stage"
    fi

    if [[ $rc -ne 0 || ! -s "$archive" ]]; then
        rm -f "$archive"
        msg_err "Archive creation failed for '$name'."
        log "ERROR" "Backup of $name failed: tar error"
        return 1
    fi

    # 2. Store SHA-256 checksum for later integrity verification
    (cd "$dest" && sha256sum "$(basename "$archive")" > "$(basename "$archive").sha256")
    chmod 640 "$archive" "$archive.sha256"

    msg_ok "Backup created: $(basename "$archive") ($(du -h "$archive" | cut -f1))"
    log "SUCCESS" "Backup created: $(basename "$archive") size=$(du -h "$archive" | cut -f1)"
}

backup_all() {
    if [[ ! -s "$CLIENT_FILE" ]]; then
        msg_err "No clients registered."; return 1
    fi
    local ok=0 fail=0 n
    while IFS='|' read -r n _ _; do
        if backup_client "$n"; then ok=$((ok + 1)); else fail=$((fail + 1)); fi
    done < "$CLIENT_FILE"
    echo "Summary: $ok succeeded, $fail failed."
}

list_backups() {
    echo "----- Backups in Central Repository -----"
    local found=0 f
    for f in "$BACKUP_ROOT"/*/*.tar.gz; do
        [[ -e "$f" ]] || continue
        found=1
        printf '%-12s %-34s %6s  %s\n' "$(basename "$(dirname "$f")")" "$(basename "$f")" \
               "$(du -h "$f" | cut -f1)" "$(date -r "$f" '+%Y-%m-%d %H:%M')"
```

```
    done
    (( found )) || msg_warn "Repository is empty."
}
```

C.5 Restore and Verify — lib/restore.sh

```
#!/bin/bash
# restore.sh - integrity verification and recovery

verify_archive() {
    local archive="$1"
    if [[ ! -f "$archive.sha256" ]]; then
        msg_err "Checksum file missing for $(basename "$archive")."
        return 1
    fi
    if ! (cd "$(dirname "$archive")" && sha256sum -c --quiet "$(basename "$archive").sha256" 2>/dev/null); then
        msg_err "Checksum MISMATCH - $(basename "$archive") is corrupted."
        log "ERROR" "Verification failed: $(basename "$archive")"
        return 1
    fi
    if ! tar -tzf "$archive" >/dev/null 2>&1; then
        msg_err "Archive structure is damaged."
        log "ERROR" "Archive unreadable: $(basename "$archive")"
        return 1
    fi
    msg_ok "Integrity verified: $(basename "$archive")"
    log "INFO" "Verification passed: $(basename "$archive")"
}

verify_backup() {
    echo "----- Verify Backup Integrity -----"
    choose_client  || return 1
    choose_archive "$SELECTED" || return 1
    verify_archive "$ARCHIVE"
}

restore_backup() {
    echo "----- Restore Backup -----"
    choose_client  || return 1
    local client="$SELECTED"
    choose_archive "$client" || return 1
    local archive="$ARCHIVE"

    verify_archive "$archive" || { msg_err "Restore cancelled (verification failed)."; return 1; }

    local default="$BASE_DIR/restored/${client}_$(date '+%Y%m%d_%H%M%S')"
    read -r -p "Restore destination [$default]: " dest
    dest="${dest:-$default}"

    if [[ "$dest" != /* ]]; then
        msg_err "Destination must be an absolute path."; return 1
    fi
    if [[ -e "$dest" && -n "$(ls -A "$dest" 2>/dev/null)" ]]; then
        msg_err "Destination '$dest' is not empty. Choose another location."; return 1
    fi
    if ! mkdir -p "$dest" 2>/dev/null; then
        msg_err "Cannot create '$dest' (permission denied)."; return 1
    fi

    if tar -xzf "$archive" -C "$dest" 2>/dev/null; then
        local count
        count=$(find "$dest" -type f | wc -l)
        msg_ok "Restored $count file(s) to: $dest"
        log "SUCCESS" "Restored $(basename "$archive") -> $dest ($count files)"
    else
        msg_err "Extraction failed."
        log "ERROR" "Restore failed: $(basename "$archive")"
        return 1
    fi
}
```

C.6 Maintenance Functions — lib/maintenance.sh

```
#!/bin/bash
# maintenance.sh - retention, storage usage, logs, reports and scheduling

apply_retention() {
    echo "----- Retention Policy (older than $RETENTION_DAYS days) -----"
    local old
    old=$(find "$BACKUP_ROOT" -name '*.tar.gz' -mtime +"$RETENTION_DAYS" | wc -l)
    if (( old == 0 )); then
        msg_info "No backups older than $RETENTION_DAYS days."; return 0
    fi
    read -r -p "Delete $old old backup(s)? [y/N]: " ans
    if [[ "$ans" =~ ^[Yy]$ ]]; then
        find "$BACKUP_ROOT" -name '*.tar.gz' -mtime +"$RETENTION_DAYS" -print -delete |
            while read -r f; do rm -f "$f.sha256"; done
        msg_ok "$old old backup(s) removed."
        log "INFO" "Retention removed $old backup(s)"
    else
        msg_warn "Cleanup cancelled."
    fi
}

storage_usage() {
    echo "----- Repository Storage Usage -----"
    df -h "$BACKUP_ROOT" | awk 'NR==1 || NR==2'
    echo
    printf '%-14s %-8s %s\n' "CLIENT" "BACKUPS" "SIZE"
    local d
    for d in "$BACKUP_ROOT"/*/; do
        [[ -d "$d" ]] || continue
        printf '%-14s %-8s %s\n' "$(basename "$d")" \
            "$(ls -1 "$d"*.tar.gz 2>/dev/null | wc -l)" "$(du -sh "$d" | cut -f1)"
    done
    echo "Free space: $(free_percent)%  (minimum required: ${MIN_FREE_PERCENT}%)"
}

view_logs() {
    echo "----- Last 15 Log Entries -----"
    if [[ -s "$LOG_FILE" ]]; then tail -n 15 "$LOG_FILE"; else msg_warn "Log file is empty."; fi
}

generate_report() {
    local report="$BASE_DIR/reports/backup_report.txt"
    {
        echo "============================================"
        echo "  CENTRALIZED BACKUP AND RECOVERY REPORT"
        echo "============================================"
        echo "Generated      : $(date '+%Y-%m-%d %H:%M:%S')"
        echo "Repository     : $BACKUP_ROOT"
        echo "Clients        : $(wc -l < "$CLIENT_FILE")"
        echo "Total backups  : $(find "$BACKUP_ROOT" -name '*.tar.gz' | wc -l)"
        echo "Repository size: $(du -sh "$BACKUP_ROOT" | cut -f1)"
        echo "Free space     : $(free_percent)%"
        echo "Successful ops : $(grep -c 'SUCCESS' "$LOG_FILE")"
        echo "Failed ops     : $(grep -c 'ERROR' "$LOG_FILE")"
        echo "--------------------------------------------"
        echo "Recent log entries:"
        tail -n 5 "$LOG_FILE"
        echo "============================================"
    } > "$report"
    msg_ok "Report saved: $report"
    cat "$report"
}

schedule_backup() {
    echo "----- Schedule Daily Automatic Backup -----"
    if ! command -v crontab >/dev/null 2>&1; then
        msg_err "crontab not found (sudo apt install cron)."; return 1
    fi
    read -r -p "Hour of day to run backup (0-23): " h
    if ! [[ "$h" =~ ^[0-9]+$ ]] || (( h > 23 )); then
        msg_err "Invalid hour '$h'."; return 1
    fi
    local job="0 $h * * * $BASE_DIR/bin/backup_server.sh --all >> $BASE_DIR/logs/cron.log 2>&1"
    if crontab -l 2>/dev/null | grep -qF "backup_server.sh --all"; then
        msg_warn "A scheduled backup already exists. Remove it with 'crontab -e' first."; return 1
    fi
    ( crontab -l 2>/dev/null; echo "$job" ) | crontab -
    msg_ok "Daily backup scheduled at $(printf '%02d' "$h"):00."
    log "INFO" "Cron job installed for hour $h"
}
```

C.7 Configuration — config/server.conf

```
# Centralized Backup and Recovery Server - configuration
BACKUP_ROOT="$BASE_DIR/repository"   # central repository for all backups
RETENTION_DAYS=7                    # delete backups older than this
MIN_FREE_PERCENT=10                 # refuse to back up if free space is lower
```

Code Explanation (Step by Step)

1. Start-up: backup_server.sh finds its own folder, loads the five function modules with `source` and stops with an error if any is missing. common.sh loads server.conf, creates the needed folders and defines helper functions.

2. Menu loop: a `while true` loop prints the menu, reads the choice with `read` and uses a `case` statement to call the matching function. Any other value reaches the `*)` branch and prints an error.

3. Register client: the name is checked with a regular expression, duplicates with `grep`, and the path with `[[ -d ]]` and `[[ -r ]]`; only then is a line appended to clients.list.

4. Backup: the function checks the client, calculates free space with df and awk, builds a time-stamped name, runs tar, checks the exit status, then writes a checksum and sets mode 640.

5. Verify and restore: `sha256sum -c` and `tar -tzf` validate the archive; restore refuses a non-empty or non-absolute destination and then extracts with tar.

6. Retention, usage, logs, report: find with `-mtime` selects old backups; du and df show usage; grep -c counts log entries for the report.

7. Scheduling: option 12 validates the hour, checks that crontab exists, avoids duplicates and installs a job that calls `backup_server.sh --all`.